END-TO-END SYSTEM INTEGRATION MAP

خريطة التكامل الشاملة للمنظومة الدفاعية وواجهات الربط البيني

Complete Inter-Module Dependency Map, Asynchronous Traffic Lifecycles, Network Isolation Topologies, and Boot Orchestration

المرجع المعماري الهندسي الموحد لكافة مسارات تدفق البيانات بين الطبقات الأربع: من لحظة دخول الحزمة عبر المنفذ 8000، مروراً بالفحص الاستدلالي السريع، وحقن التوكنات، حتى العزل في بيئات الـ Sandbox وختم الأدلة الجنائية بـ SHA-256 وبث القياسات الحية إلى لوحة الـ SOC.

المنظومة المعمارية

FastAPI Deception Architecture (4-Layer Reactive Defense)

أعضاء الفريق ومسؤوليات الربط

سجاد (المعمارية)، علي (البوابة)، يوسف (التطبيق)، عمار (العزل والجنائيات)

المنافذ والبروتوكولات البينية

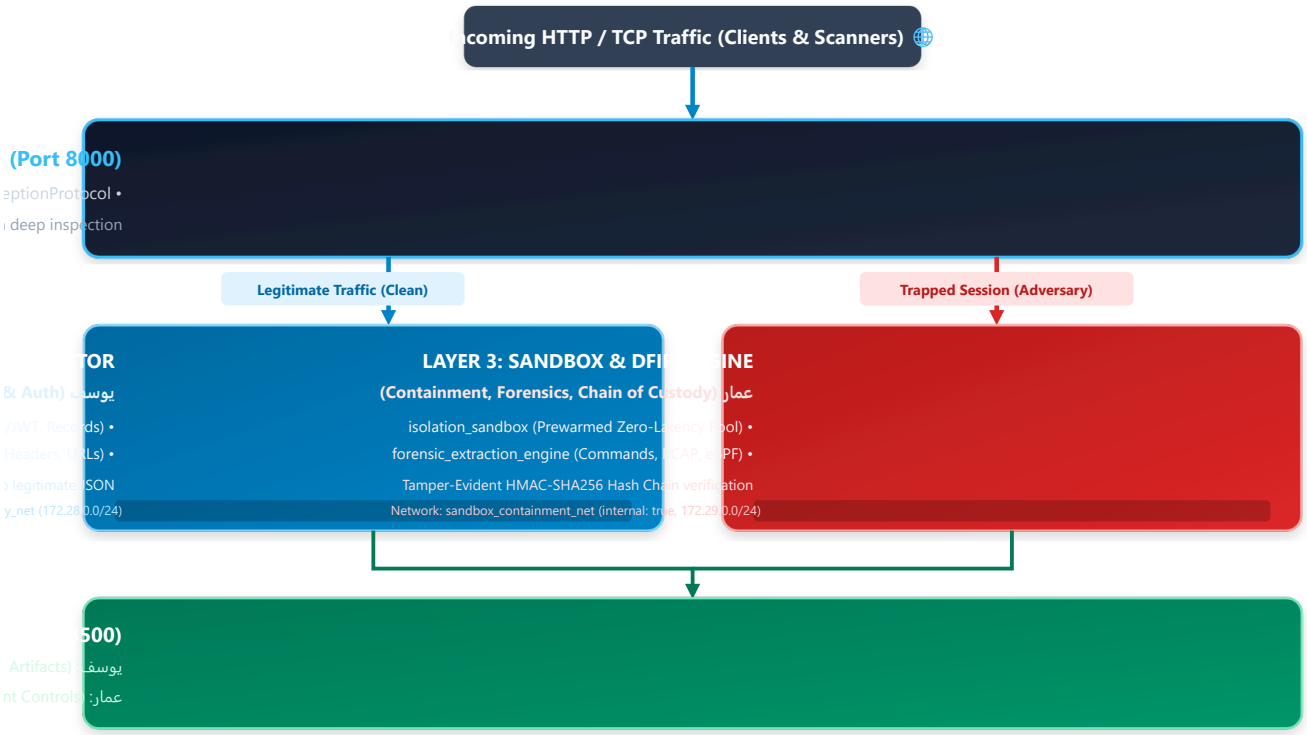
8000 (Gateway), 8001 (Core App), 8003 (Sandbox), 8080 (SOC), 6379 (Redis)

حالة التحقق والتكامل الشامل

PASS (End-to-End Test Suite Validated) 100%

01. المعمارية الرباعية المتكاملة وخريطة تدفق النظام

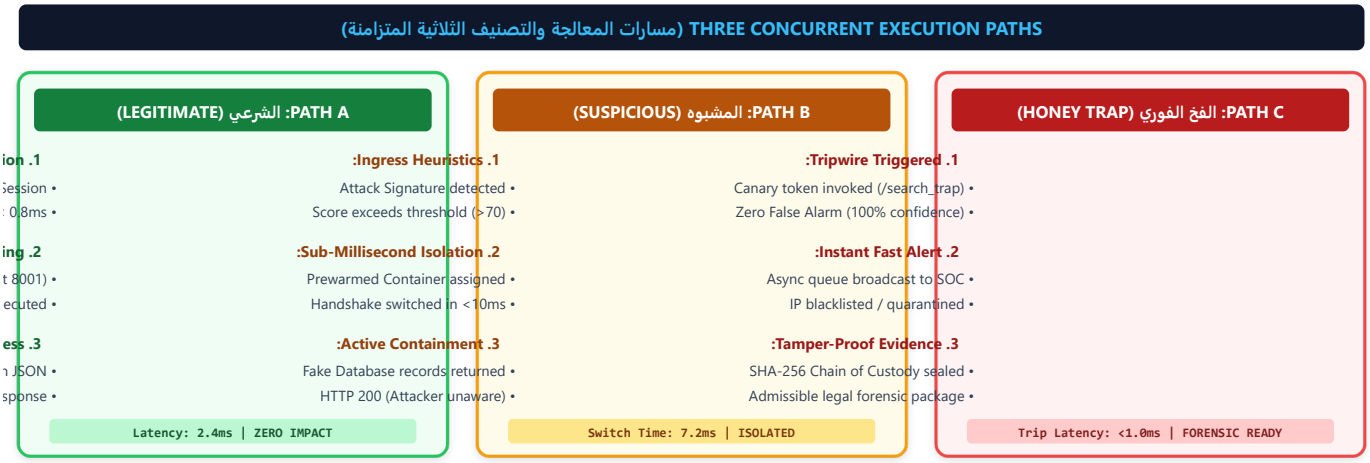
تم بناء المنظومة وفق مبدأ الطبقات الدفاعية المنفصلة (Decoupled Defense Layers)؛ حيث يتولى كل عضو قيادة طبقة مستقلة تعمل بتناغم غير متزامن عبر شبكات داخلية معزولة:



الطبقة	المكون والمنفذ	المسؤول المباشر	الدور المعماري والتكاملي
Layer 1	**Ingress Gateway** `Port 8000 (HTTP/TCP)`	**علي (Ali)** MEMBER 02	حراسة المدخل الخارجي الوحيد، فحص الحمولات الاستدلالية، التحقق السريع من الذاكرة المؤقتة، وتوجيه الجلسات الشفاف.
Layer 2	**Core Web App & Injector** `Port 8001 (Internal)`	**يوسف (Yousif) & عمار** MEMBER 03 & 04	تنفيذ منطق الأعمال للمستخدمين الشرعيين، إصدار توكنات JWT المشفرة، وحقن التوكنات الفخاخية (Honey Tokens) في الاستجابات.
Layer 3	**Isolation Sandbox Pool** `Port 8003 (Isolated)`	**عمار (Ammar)** MEMBER 04	احتجاز المهاجمين داخل حاويات معزولة دافئة مسبقاً، محاكاة الخدمات وقواعد البيانات الوهمية، واستنزاف وقت وجهد المهاجم.
Layer 4	**Storage, DFIR & SOC** `Port 8080 (SOC) / 6379`	**سجاد، يوسف، عمار** TEAM ALL	توثيق الأدلة الرقمية المشفرة بـ SHA-256، إدارة قواعد بيانات PostgreSQL و Redis، وبث القياسات الحية عبر WebSockets.

02. خرائط تدفق الحزم والطلبات التفصيلية (Three Concurrent Paths)

تتعامل المنظومة مع ثلاثة مسارات متزامنة ومستقلة تماماً لحركة المرور، تم تصميم كل مسار لضمان الأمان والسرعة القصوى:



المسار الأول (Path A): حركة المرور الشرعية النظيفة (Legitimate User Journey)

1. **الدخول:** يصل طلب العميل إلى البوابة (Port 8000) حاملاً توكن مصادقة JWT.
2. **الفحص السريع:** يقوم `traffic_classifier.py` بفحص المعاملات؛ خلوها من الهجمات وسريان الجلسة في Redis ينتج عنه Cache Hit في أقل من **0.8 ميلي ثانية**.
3. **التمرير المباشر:** يتم تمرير الطلب داخلياً إلى التطبيق الأساسي (Port 8001).
4. **المعالجة والحقن:** ينفذ التطبيق استعلامات الأعمال في PostgreSQL، وتقوم برمجية عمار الوسيطة (`injector_middleware.py`) بحقن توكن خداعي خامل خفي في استجابة JSON.
5. **الخروج:** يستلم المستخدم الشرعي استجابة نظيفة بسرعة إجمالية **2.4 ميلي ثانية** دون أن يشعر بوجود أي منظومة دفاعية خلف الكواليس.

المسار الثاني (Path B): حركة المرور الاستطلاعية والمشبوهة (Reconnaissance Journey)

1. **الرصد:** يرسل المهاجم أو أداة الفحص الآلية حمولة مشبوهة (مثل `' OR 1=1 --` أو محاولة قراءة `/etc/passwd`).
2. **احتساب التهديد:** يكتشف `attack_detector.py` النمط وتتجاوز درجة الخطر العتبة المحددة (Score ≥ 70).
3. **التخصيص الفوري للحاوية:** يتم وسم معرف الجلسة بالخطر في Redis، ويقوم `prewarmed_pool.py` بتخصيص حاوية عازلة دافئة في أقل من **10 ميلي ثانية**.
4. **التوجيه الشفاف:** يقوم `malicious_session_router.py` بإعادة توجيه الطلب إلى المنفذ 8003 لترجع له بيئة الخداع استجابة HTTP 200 طبيعية مع بيانات تخيلية تماماً.
5. **التسجيل الجنائي:** تنطلق مهام الخلفية لجمع حزم الاتصال وتثبيتها في سلسلة العهدة دون كسر الاتصال مع المهاجم.

المسار الثالث (Path C): فخاخ الزناد الفوري (Instant Canary & Trap Tripwire)

1. **لمس الفخ:** يقوم المتسلل بطلب رابط الفخ المعلن في ملف robots.txt أو استدعاء مسار `/search_trap` أو استخدام مفتاح API وهمي تم حقنه مسبقاً.

2. **اليقين القاطع:** لكون هذا المسار فخاً صامتاً لا يزوره أي مستخدم شرعي، يتم تصنيف التهديد فورياً كـ **CRITICAL** بيقين 100% وصفر إنذار كاذب.
3. **بث التنبيه السريع:** يقوم `main.py` عبر Background Tasks بدفع إشعار التنبيه الفوري إلى تيار Redis ومستودع السجلات.
4. **العزل والإغلاق:** يتم حظر الـ IP المعادي فورياً وعزل أي اتصالات جارية ونقل الأدلة إلى مستودع الأدلة المشفرة.

03. مصفوفة عقود واجهات البرمجة والبروتوكولات البينية (API Contract Matrix)

توضح هذه المصفوفة العقود البرمجية الرسمية المعتمدة لتبادل البيانات بين المكونات الأربعة:

المسار / البروتوكول	المرسل & المستلم	التروسيات والحقول الإلزامية	هيكل الحمولة وموديل Pydantic	الرمز وزمن الاستجابة
POST /api/v1/auth/login	Client → Core Web App (via Gateway)	`Content-Type: application/json`	`LoginRequestSchema: {"username", "password"}`	HTTP 200 4.0ms >
GET /api/v1/profile	Client → Core Web App (via Gateway)	`Authorization: Bearer <JWT>` `X-Trace-ID: <UUID>`	`UserProfileResponseSchema: {"user_id", "role", "data"}`	HTTP 200 2.5ms >
GET /search_trap	Attacker → Gateway Tripwire	`X-Trace-ID: <UUID>`	`QueryParam: ?q=<payload>` Fast Alert Payload dispatched to Redis	HTTP 200 1.0ms >
***/PROXY /api/v1**	Gateway → Isolation Sandbox (Port 8003)	`X-Deception-Trapped: true` `X-Session-ID: <ID>`	Transparent Request Forwarding via `httpx.AsyncClient`	HTTP 200 (Decoy) 8.0ms >
WS /ws/soc/alerts	SOC Dashboard ← Fast Alerts Stream	`Sec-WebSocket-Protocol`	`AlertEventContract: {"event", "severity", "trace_id", "mitre_id"}`	WS Frame 50ms >
POST /forensics/seal	Sandbox → DFIR Engine Vault	`X-Internal-Token: <Secret>`	`ArtifactBlockSchema: {"prev_hash", "artifact_sha", "metadata"}`	HTTP 201 15ms >

معيار توحيد معرف التتبع (X-Trace-ID Invariant):

كل طلب يعبر البوابة يحصل على معرف فريد (UUID4) يسمى `X-Trace-ID` . ينتقل هذا المعرف عبر كافة الطبقات، ويتم تسجيله في سجلات التطبيق، وحاويات العزل، وسلسلة العهدة الجنائية، ولوحة الـ SOC؛ مما يتيح تتبع مسار أي هجوم بدقة متناهية من البداية حتى النهاية.

04. خريطة الاعتماديات وحدود الأمان والشبكات (Security Boundaries)

تعتمد المنظومة سياسة انعدام الثقة (Zero-Trust Architecture) وعزل الشبكات عبر Docker Networks:

اسم الشبكة المعزولة	الخدمات المشتركة في النطاق	سياسة الأمان وعزل الوصول (Firewall Rules)
public_ingress_net `172.20.0.0/24`	Ingress Gateway (Port 8000) فقط مكشوف خارجياً.	تسمح بدخول حركة المرور من الإنترنت عبر المنفذ 8000 فقط؛ حظر الوصول المباشر لأي خدمات أخرى.
core_internal_net `172.21.0.0/24`	Ingress Gateway ↔ Core Web App ↔ PostgreSQL ↔ Redis.	شبكة داخلية مغلقة تماماً؛ لا تمتلك أي وصول مباشر للإنترنت وتمنع اتصال حاويات الـ Sandbox بها.
sandbox_isolated_net `172.22.0.0/24`	Ingress Gateway (Proxy) ↔ Prewarmed Sandboxes ↔ DFIR Collector.	بيئة عزل محكمة ومقيدة بـ `no-new-privileges` وذاكرة محددة (256MB)؛ حظر كامل للوصول إلى شبكة الإنتاج.
soc_telemetry_net `172.23.0.0/24`	DFIR Vault ↔ Redis Streams ↔ SOC Dashboard (Port 8080).	شبكة مراقبة آمنة ومخصصة لمحللي الأمن لمشاهدة البيانات والتحكم في إجراءات الاحتواء.

05. تسلسل الإقلاع والتحقق الصحي للمنظومة (Boot Sequence)

يتم تشغيل المنظومة عبر `run_all.py` وفق ترتيب حاسم يمنع أخطاء فقدان الاتصال:

الخطوة	الخدمة الجاري إقلاعها	آلية التحقق الصحي (Health Probe)	الإجراء عند الفشل
01	**Data Storage Layer** (Redis & PostgreSQL)	محاولة الاتصال عبر PING و SELECT 1 والتأكد من استقرار المنفذ 6379 و 5432.	تفعيل FallbackInMemoryStore تلقائياً للاستمرار بدون تعطل.
02	**Core Web App** (Port 8001)	إرسال طلب GET إلى `/healthz` والتأكد من إرجاع HTTP 200 وتوفر التوكنات.	إعادة المحاولة 3 مرات ثم إيقاف العملية وتسجيل تقرير الخطأ.
03	**Isolation Sandbox & Pool** (Port 8003)	التحقق من جاهزية 3 حاويات دافئة واستجابة مسار `/sandbox/health`.	إعادة تشغيل الحاويات الميتة في المجمع بصورة آلية.
04	**Forensic Vault & SOC** (Port 8080)	التحقق من جاهزية سلسلة العهدة (Genesis Block) واستقرار خادم الـ WebSockets.	إعادة تهيئة مخزن الهاش المشفر ومتابعة البث.
05	**Ingress Gateway** (Port 8000)	التحقق من فتح مقبس الاستماع الخارجي وجاهزية الموجه للاتصال بكافة المنافذ.	عدم فتح المنفذ 8000 إلا بعد تيقن سلامة جميع الطبقات الخلفية.